Supervisor Lab — Troubleshooting Guide

Symptom → diagnostic → fix for the failure modes we've actually hit. Each section is self-contained; jump to the one that matches your error.

Networking

VIP pings fail from outside the workload subnet

Symptom

```
Verifying VIPs are reachable (catches Phase 11 regression)...
  192.168.3.249 DEAD (Phase 11 fix may not have run)
```

…even though the HAProxy VM has all six VIPs bound. Sometimes one or two VIPs respond, the rest don't.

Diagnostic

```
# Are the VIPs actually on the VM?
ssh ubuntu@192.168.3.245 'ip -4 addr show ens192'
# Look for inet 192.168.3.249/32, .250/32, … all six.

# Single ping with a long timeout
ping -c 1 -W 10 192.168.3.250
# If this succeeds (even after a 4–8 s wait), VIPs are bound and routing
# is fine — the problem is first-packet ARP latency.

# tcpdump on the VM to watch the ARP exchange:
sudo tcpdump -i ens192 -n 'arp or icmp'
```

Cause

Linux's default `arp_announce` doesn't gratuitously announce secondary `/32` IPs. The first packet from a host outside the VM's subnet has to wait for the upstream router's ARP-Who-Has → reply cycle. On this lab that takes 5–8 s per VIP, longer than the validate script's default ping timeout.

Fix (durable)

`modules/haproxy/templates/user-data.yaml.tpl` runs `arping -A` for every VIP at boot — primes upstream caches before the validator runs. If you're testing manually and see DEAD, re-run with a longer `-W` (`-W 10` is plenty).

ARP cache and routed destinations

If you `arp -na | grep 192.168.3` on your Mac and see nothing, **that's expected** — your Mac isn't on `192.168.3.0/24`.

ARP only resolves IPs to MACs for hosts that share an L2 broadcast domain. For destinations on a different subnet, your Mac sends packets to the default gateway with the gateway's MAC; the gateway is the one doing ARP toward the target. So your ARP table only ever holds entries for:

- Hosts on your own subnet
- Your default gateway

Trace it yourself:

```
ifconfig en0 | grep "inet "          # your local IP + mask
route -n get default                  # default gateway
arp -na | grep <gateway-ip>           # gateway's MAC should be cached
```

Take-away: slow first-packet behavior for routed destinations is typically the **router's** ARP toward the target, not yours.

Static IP doesn't take after cloud-init

Symptom

Nested ESXi or HAProxy/NFS VM comes up on a DHCP-assigned IP (e.g. `192.168.3.21`) instead of the configured static.

Diagnostic

```
ssh ubuntu@<dhcp-ip>                  # connect to whatever IP it got
ip -4 addr show ens192                # see which IPs are bound
ls /etc/netplan/                      # 60-static.yaml should be present
sudo cat /etc/netplan/60-static.yaml  # is the file's YAML structure correct?
sudo netplan apply                    # manually re-apply
```

Cause / Fix

The netplan we generate via `templatefile()` requires careful indentation when interpolating multi-line strings. In `modules/haproxy/main.tf` the `netplan_vip_addresses` local must produce lines prefixed with **14 spaces** so they nest correctly under `addresses:` in the rendered block-

scalar. If you tweak that local, render the cloud-init by hand (`terraform console` → `local.cloud_init`) and inspect the netplan section.

DNS resolution

DNS issues in this lab cascade hard — the same hostname can route to vCenter, to the Supervisor (via the router's port-forward), or to nothing at all, depending on whose DNS chain you're in. These symptoms typically point at DNS:

- `kubectl vsphere login` succeeds at password prompt then dies with "Error while getting list of workloads: internal server error" (the wcp-login service inside the CP VMs can't validate SSO against vCenter)
- govc commands return `POST "/sdk": 404 Not Found` (govc hitting the Supervisor's K8s API instead of vCenter's SOAP endpoint)
- Supervisor enable bails on "vCenter certificate is invalid" at the "Configured Control Plane VMs" step
- terraform destroy's REST-based provisioner returns garbage from `/api/session` and the script's hex-token sanity check rejects it

macOS — what your Mac actually resolves to

`dig` and `curl` can disagree on macOS because they take different resolution paths. `dig` queries the configured nameserver directly; `curl` (and browsers, kubectl, ssh) go through macOS's system resolver chain.

```
# 1. Which resolvers does the SYSTEM use (the path curl takes)?
scutil --dns
# Look for "nameserver[0]" under each "resolver #" block. The order
# matters — supplemental resolvers can override the default for
# specific domains (VPN tools, MagicDNS, etc).

# 2. What does the system resolver actually return RIGHT NOW?
dscacheutil -q host -a name vcenter.skynetsystems.io
# Returns the same answer curl/kubectl will use. Compare with dig:
dig +short vcenter.skynetsystems.io
# If they disagree, something between the configured nameserver and
# the system resolver is intercepting (DoH, iCloud Private Relay,
# MDM profile, NextDNS, etc.).

# 3. Flush macOS DNS cache (cheap to try first):
sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder

# 4. Force a specific IP for a hostname (bypasses everything for
#    this one curl call — use when chasing routing issues):
curl --resolve vcenter.skynetsystems.io:443:192.168.2.80 \
     https://vcenter.skynetsystems.io/api/session
```

```
# 5. ARP cache (only entries for hosts on your own subnet appear;
#    for routed destinations, you'll only see the gateway's MAC):
arp -na
route -n get default     # which gateway is "default" for the Mac
```

Linux (CP VMs, Photon, Ubuntu) — what systemd-resolved returns

CP VMs use systemd-resolved. Its stub listens on `127.0.0.53`, and its actual resolution path can differ from what `dig` against an explicit server returns.

```
# 1. Full systemd-resolved status: per-link DNS servers, search
#    domains, DNSSEC/DoT/mDNS state.
resolvectl status

# 2. What systemd-resolved returns (the path curl/wcp-login takes):
resolvectl query vcenter.skynetsystems.io
# The "-- link: ethX" annotation tells you which interface's DNS
# server gave the answer. The "(other.domain.tld)" in parens
# indicates a CNAME chain — see the +short gotcha below.

# 3. NSS path (consults /etc/hosts BEFORE going to systemd-resolved):
getent hosts vcenter.skynetsystems.io

# 4. Flush the systemd-resolved cache (may need a couple seconds):
resolvectl flush-caches

# 5. Directly query a specific server, bypassing the stub:
dig @192.168.1.1 vcenter.skynetsystems.io
# Compare with `getent hosts` above. If they disagree, you have a
# CNAME chain or a misbehaving resolver — see next subsection.

# 6. Where did curl actually connect?
curl -sk -o /dev/null -w 'remote=%{remote_ip}\n' \
     https://vcenter.skynetsystems.io/healthz
```

Gotcha: `dig +short` hides CNAMEs

`dig +short hostname` shows only the final A record in any CNAME chain — it silently follows intermediate CNAMEs. This makes a broken split-horizon DNS setup look fine when it isn't:

```
# A resolver might be returning a CNAME chain like:
#   vcenter.skynetsystems.io   CNAME  skynetsystems.io
#   skynetsystems.io           A      136.47.236.42   (public — no internal
        override)

# `dig +short` returns just the final IP:
dig +short @192.168.1.1 vcenter.skynetsystems.io
# → 192.168.2.80               ← looks fine if you have an override on the first
        name
```

```
# But the full output shows the CNAME, and that's where the bug lives:
dig @192.168.1.1 vcenter.skynetsystems.io
# → ;; ANSWER SECTION:
#   vcenter.skynetsystems.io.  CNAME  skynetsystems.io.
#   skynetsystems.io.          A      136.47.236.42  ← the apex returns public!

# systemd-resolved follows the CNAME, ends up at the apex, gets the
# public IP, hands that back. dig +short happened to not show the
# CNAME so the bug stayed hidden until we ran the full query.
```

Rule of thumb: when DNS is suspect, drop `+short` from `dig`. The CNAME chain often is the actual problem.

Workarounds vs. proper fixes

When DNS is broken, three options in order of cleanliness:

Approach	Scope	Survives rebuild?
Add the missing record at the upstream DNS server (e.g. an A record for the apex domain pointing to the right IP)	Entire network	Yes
`/etc/hosts` entry on the affected hosts (`192.168.2.80 vcenter.skynetsystems.io`)	One host	No — vSphere may rotate CP VMs
`curl --resolve` per-command bypass	One invocation	n/a — for diagnostics

In this lab, the Supervisor's CP VMs hitting the wrong endpoint was traced to the **apex** `skynetsystems.io` lacking an internal A record. The CNAME chain `vcenter → skynetsystems.io → public IP` was correctly returned by the LAN DNS, but the chain ended at the public IP. Adding an A record for `skynetsystems.io` → `192.168.2.80` at the LAN's authoritative resolver fixed it for every host on the network — much cleaner than per-host `/etc/hosts` patches.

CP VM-specific debugging

Get a shell on a CP VM (the wcp service runs there, not on the vCSA):

```
# Get the current CP VM root password (rotates frequently):
expect <<'EXP' | grep -E '^(IP|PWD):'
log_user 0
spawn ssh root@192.168.2.80
expect "assword:"; send "<vCSA-root-pw>\r"
```

```
expect "Command>"; send "shell\r"
expect "ot@vcenter"
log_user 1
send "/usr/lib/vmware-wcp/decryptK8Pwd.py | grep -E '^(IP|PWD):'\r"
expect "ot@vcenter"
log_user 0
send "exit\r"; expect "Command>"; send "exit\r"; expect eof
EXP

# Then SSH into the CP VM and test resolution:
sshpass -p '<cp-pw>' ssh root@<cp-ip>
# Inside: run the resolvectl/dig/getent commands above
```

OVF Deploy / `vsphere_virtual_machine` quirks

"data center ID is required for ovf deployment"

Cause: the provider needs `datacenter_id` set on the resource itself (not just inferred through `resource_pool_id`) when `ovf_deploy {}` is used.

Fix: add `datacenter_id = var.datacenter_id` to the `vsphere_virtual_machine` block.

"this virtual machine requires a client CDROM device to deliver vApp properties"

Cause: the provider's plan-time check on OVF-deployed VMs assumes vApp properties get delivered via a CDROM drive, even when you're using `extra_config` / `guestinfo` instead. Without a CDROM stanza, refresh fails on subsequent applies.

Fix: add a no-op CDROM block to satisfy the check:

```
cdrom {
  client_device = true
}
```

OVF-deployed VM stays powered off

Cause: when `ovf_deploy {}` is used, the vmware/vsphere provider's import path doesn't auto-power-on the way native VM creation does.

Diagnostic: the apply hangs at `Still creating... [Xm Ys elapsed]` and `govc vm.info <vm>` shows `Power state: poweredOff`.

Fix: explicit power-on via a follow-up `null_resource` (already done in modules/haproxy and modules/nfs). Re-applies are idempotent — script detects existing `poweredOn` state and skips:

```
command = "if govc vm.info ${var.vm_name} 2>/dev/null | grep -q 'Power
state:.*poweredOn'; then echo 'already powered on'; else govc vm.power -on=true $
{var.vm_name}; fi"
```

If `vm.info -json` is preferred, beware: `govc -json` adds whitespace around the colon, so `grep -o '"powerState":"…"'` (no space) silently returns empty. Either tolerate the space with a regex, or stick with text `vm.info` output.

"The attempted operation cannot be performed in the current state (Powered on)"

Cause: Terraform tried to reconfigure something on a running VM (e.g. add a CDROM in-place). The vSphere provider doesn't always auto-power-off + reconfig + power-on cleanly.

Fix: destroy + recreate the VM:

```
govc vm.power -off=true -force=true <vm>
govc vm.destroy <vm>
terraform state rm 'module.<...>.vsphere_virtual_machine.<vm>'
terraform apply
```

"The virtual machine is not supported on the target datastore"

Cause: you pointed the VM resource at a datastore that the chosen resource pool's hosts can't see. In this lab: putting HAProxy/NFS on the Supervisor cluster's resource pool but with `datastore1` (which is local to the *physical* host, not visible to nested ESXi).

Fix: for outer-management VMs (haproxy, nfs), use the **physical** cluster's resource pool:

```
resource_pool_id = data.vsphere_compute_cluster.physical.resource_pool_id
```

Don't conflate cluster_id with resource_pool_id — the cluster moref (`domain-c130`) is not a valid `resource_pool_id` (which expects `resgroup-NNN`).

`vsphere_tag` / `vsphere_nas_datastore` race

Symptom: `null_resource.nfs_shared_tag` completes in 1 s but a later `govc tags.attached.ls supervisor-storage` returns empty. The Supervisor enable then complains:

```
Failed to create Default Kubernetes Content Library because Namespaces
No compatible datastore matching given storage policy: <uuid>.
```

Cause: `vsphere_nas_datastore` returns "Creation complete" before vSphere's tag-attachable inventory has indexed the new datastore. The subsequent `govc tags.attach` succeeds at the API level but the tag silently doesn't stick.

Fix: the tag-attach `null_resource` now resolves the datastore's moref and **verifies** the tag is visible via `tags.attached.ls`, with retries. If you ever hit this manually:

```
govc tags.attach supervisor-storage /Datacenter/datastore/nfs-shared
govc tags.attached.ls supervisor-storage    # must show Datastore:datastore-NNN
```

Once the tag IS attached, vSphere's reconcile loop picks it up within ~1–2 min and Supervisor continues progressing.

Gotcha: `govc object.collect -s` vs `govc ls -i` for morefs

Earlier the verify loop above silently broke because the script used `govc object.collect -s <path> | head -1` to grab the moref. That command dumps *every property* of the managed object, one per line; the first line is whatever property happens to be serialized first — on a freshly-mounted datastore it's a literal `true` (a boolean property). So the script ended up comparing `Datastore:true` against `tags.attached.ls` output and never matched, even when the tag was attached.

```
# WRONG — returns the value of a property, not the moref:
govc object.collect -s /Datacenter/datastore/nfs-shared | head -1
# → true

# RIGHT — `ls -i` prefixes each line with the moref:
govc ls -i /Datacenter/datastore/nfs-shared
# → Datastore:datastore-1060 /Datacenter/datastore/nfs-shared

# Extract just the moref:
govc ls -i /Datacenter/datastore/nfs-shared | awk '{print $1}'
# → Datastore:datastore-1060
```

Rule of thumb: `object.collect` **is for property values,** `ls -i` **is for morefs.** If you need both the moref and a single property, `govc ls -l -i <path>` shows type + moref + path, or use `object.collect -s <path> <propertyName>` with an explicit property.

Gotcha: `null_resource` doesn't re-run when the script changes

If you fix a bug in the `local-exec` `command` of a `null_resource`, **Terraform won't re-run it on the next apply** — `null_resource` only re-executes when a value in its `triggers` block changes. The provisioner's script content isn't part of triggers; it's part of the resource body, which Terraform doesn't fingerprint for re-execution.

This means:

1. You ship a bug in a `null_resource` script.
2. The script runs, "succeeds" (or silently no-ops, like the moref bug above), and goes into state.
3. You fix the script in HCL.
4. `terraform apply` shows "no changes" for that resource — Terraform considers the resource up-to-date because its triggers haven't moved.
5. Bug persists in the live infrastructure forever.

Three ways to handle it:

```
# Option A: add an explicit revision string to triggers. Bump it on edits.
triggers = {
  …existing keys…
  script_rev = "2026-05-25-moref-via-ls-i"
}

# Option B: hash the script content (works if you extract it to a local).
locals {
  attach_script = file("${path.module}/attach-tag.sh")
}
resource "null_resource" "nfs_shared_tag" {
  triggers = { script_sha = sha1(local.attach_script) }
  provisioner "local-exec" { command = local.attach_script }
}

# Option C (one-off): force re-run on this apply.
terraform apply -replace='module.…null_resource.nfs_shared_tag'
# (or `terraform taint <addr>` on TF < 0.15.2)
```

The `supervisor` module's `nfs_shared_tag` uses Option A (`script_rev`).

Supervisor enable (REST API)

govc 0.54+'s `namespace.cluster.enable` only supports NSX-T. For HAProxy / `VSPHERE_NETWORK`, hit the REST API directly.

Discovering the right spec schema

vSphere's own metamodel exposes the full enable-spec schema:

```
SESSION=$(curl -sk -u $USER:$PASS -X POST https://vcenter/api/session | tr -d '"')
curl -sk -H "vmware-api-session-id: $SESSION" \
  "https://vcenter/rest/com/vmware/vapi/metadata/metamodel/structure/
        id:com.vmware.vcenter.namespace_management.clusters.enable_spec" \
  | python3 -m json.tool | less
```

Lists every field with type, optional/required, and (if a union) under which value of which discriminator the field is allowed. Faster than guessing JSON shapes.

"An edge provider must be configured" (HTTP 400)

Cause: `load_balancer_config_spec` field is missing or named incorrectly. The correct path is `spec.load_balancer_config_spec` (singular, no wrapper) with `network_provider = "VSPHERE_NETWORK"` set at the top of `spec`.

"Failed to validate Content Library UUID , error: not found"

Cause: the spec includes `default_kubernetes_service_content_library = ""`. vSphere 9 rejects the empty string.

Fix: omit the field entirely (already done). To enable TKG later, provide a real subscribed library UUID.

"Address count 1 must be greater than or equal to 5"

Cause: `master_management_network.address_range.address_count` was too low. vSphere requires ≥ 5 IPs in the management range even for HA-off setups (CP + floating + upgrade overhead).

Fix: always set `address_count = 5` (already done).

Supervisor stuck `CONFIGURING` with `kubernetes_status: WARNING`

Often non-fatal — vSphere finishes Supervisor enable even if the default content library can't be created. Check the actual messages:

```
SESSION=$(curl -sk -u $USER:$PASS -X POST https://vcenter/api/session | tr -d '"')
curl -sk -H "vmware-api-session-id: $SESSION" \
  https://vcenter/api/vcenter/namespace-management/clusters/<cluster-moref> \
  | python3 -m json.tool
```

Look at `messages[]`. WARNINGS don't stop progression; ERROR does.

Cloud-init

`write_files` aborts on a missing user/group

Symptom: Earlier-listed files written, later ones silently absent. `/var/log/cloud-init.log` shows:

```
KeyError: "getgrnam(): name not found: 'haproxy'"
OSError: Unknown user or group: ...
```

Cause: a `write_files` entry has `owner: 'root:haproxy'` but the `haproxy` group doesn't exist yet — it's created when the `haproxy` package installs (which happens *after* `write_files`).

Fix: drop the `owner:` line and let it default to `root:root`. The target service can still read 0644 / 0600 files even if the group isn't yet created.

HAProxy

The lab actually has *two* HAProxy instances and either can break:

- **Lab HAProxy VM** (`192.168.3.245`) — built by Terraform's `haproxy` module via OVF + cloud-init. Hosts the Dataplane API and assigns the Supervisor VIPs from the pool `192.168.3.249-254`.
- **Network-edge HAProxy** (e.g. on a router/Pi at `192.168.1.10`) — routes external TLS-SNI traffic to the right internal host (vCenter, NAS, Supervisor VIP, etc.).

The diagnostic flow is the same for both.

HAProxy refuses to start (config error)

Symptom

```
systemctl status haproxy
× haproxy.service - HAProxy Load Balancer
```

```
   Active: failed (Result: exit-code) since … ; 5min ago
  Process: 1964 ExecStart=/usr/sbin/haproxy -Ws -f $CONFIG -p $PIDFILE …
(code=exited, status=1/FAILURE)
   …
   haproxy.service: Start request repeated too quickly.
```

systemd hides the actual reason behind `status=1/FAILURE` and rate-limits restart attempts ("repeated too quickly"). You need the haproxy daemon's own stderr to see what's wrong.

Diagnostic (two ways)

```
# 1. Run HAProxy's config validator directly — doesn't start the daemon,
#    just parses the file and prints [ALERT] / [WARNING] lines:
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
# Exits 0 on success, non-zero with reasons on failure. Always run this
# before `systemctl start` — fastest way to surface config bugs.

# 2. systemd journal for the haproxy unit captures the daemon's stderr
#    from each restart attempt:
sudo journalctl -u haproxy --no-pager -n 30
sudo journalctl -u haproxy -f     # follow new lines live
```

The same `[ALERT] config : …` lines appear in both outputs. `systemctl status haproxy` would also show ~10 of these journal lines, but `journalctl -u haproxy` gives you the full window.

Real example: missing port in a backend server

The alert that named the problem:

```
[ALERT] config : config: backend 'k8s': server 'k8s-server' has neither
service port nor check port nor tcp_check rule 'connect' with port information.
[ALERT] config : Fatal errors found in configuration.
```

That `[ALERT]` told us exactly where to look — `backend k8s`, server `k8s-server`. Reading the config confirmed:

```
backend k8s
        balance roundrobin
        mode tcp
        server k8s-server 192.168.3.250 check maxconn 20
                          ^^^^^^^^^^^^^ no :port — fatal
```

Every other backend in the same file had `:443` after the IP. Easy fix:

```
        server k8s-server 192.168.3.250:443 check maxconn 20
```

Other common HAProxy config failure modes

`[ALERT]` text fragment	Likely cause	Fix
`has neither service port nor check port`	`server` line missing `:port`	Add `:443` (or whichever target port) after the IP
`unknown option 'XYZ' for backend`	Option not valid in that mode (e.g. `httplog` in `mode tcp`)	Switch to the `tcp`-equivalent (`tcplog`) or remove the option
`Cannot bind to socket [0.0.0.0:443]`	Port in use by another process	`sudo ss -tlnp | grep ':443'`; stop the other service or pick a different port
`Could not open configuration file`	Wrong path passed via `-f`, or perms wrong	Check `sudo systemctl cat haproxy` for the exact `ExecStart`; verify the file exists and is readable by `haproxy` user
`error : cannot open ... certificate`	TLS cert path wrong or unreadable	`ls -la` the cert path; chown so `haproxy` user can read

After a config edit

systemd-managed haproxy is restart-rate-limited. If you fixed the config and `systemctl start` still says “Start request repeated too quickly,” clear the failure counter first:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg       # confirm valid
sudo systemctl reset-failed haproxy               # clear rate-limit
sudo systemctl start haproxy
sudo systemctl is-active haproxy                  # should print "active"
```

For zero-downtime config reloads on an already-running instance:

```
sudo systemctl reload haproxy
# Or:  sudo haproxy -W -p /run/haproxy.pid -sf $(cat /run/haproxy.pid) \
#              -f /etc/haproxy/haproxy.cfg
```

`reload` keeps existing connections draining on the old process while new connections go to the new process. `restart` drops all connections.

Lab HAProxy VM specifics

If the **lab's HAProxy VM** (`192.168.3.245`) is the one misbehaving, its config is rendered by cloud-init from `terraform/modules/haproxy/templates/user-data.yaml.tpl` at first boot. Cloud-init only runs once, so an edit to the template doesn't reach existing VMs — you'd `terraform taint module.supervisor_lab.module.haproxy.vsphere_virtual_machine.haproxy` and re-apply to get a fresh VM.

Quick check on the running HAProxy VM:

```
sshpass -p '<root-pw>' ssh ubuntu@192.168.3.245 \
    "sudo systemctl status haproxy --no-pager; \
     sudo haproxy -c -f /etc/haproxy/haproxy.cfg 2>&1 | tail -10"
```

HA alarm spam

"Insufficient heartbeat datastores"

Symptom: Every nested ESXi shows the HA red icon with:

```
The number of vSphere HA heartbeat datastores for host <ip> in cluster
Supervisor-Cluster in Datacenter is 1, which is less than required: 2
```

Cause: vSphere HA wants ≥ 2 datastores per host for heartbeat redundancy. Lab hosts only see `nfs-shared` (`vsanDatastore` is empty / unconfigured; physical-host `datastore1` isn't accessible to nested ESXi).

Fix (silence): set the cluster advanced option `das.ignoreInsufficientHbDatastore = true` (govc 0.54 cluster.change doesn't expose advanced options; use pyvmomi):

```
python3 <<'PY'
import os, ssl
from pyVim.connect import SmartConnect
from pyVmomi import vim
ctx = ssl.create_default_context(); ctx.check_hostname=False;
        ctx.verify_mode=ssl.CERT_NONE
si = SmartConnect(host=os.environ['GOVC_URL'], user=os.environ['GOVC_USERNAME'],
                  pwd=os.environ['GOVC_PASSWORD'], sslContext=ctx)
for dc in si.RetrieveContent().rootFolder.childEntity:
    for c in dc.hostFolder.childEntity:
        if c.name == 'Supervisor-Cluster':
            spec = vim.cluster.ConfigSpecEx()
            spec.dasConfig = vim.cluster.DasConfigInfo()
            spec.dasConfig.option = [vim.option.OptionValue(
                key='das.ignoreInsufficientHbDatastore', value='true')]
```

```
            c.ReconfigureComputeResource_Task(spec=spec, modify=True)
            print("set")
PY
```

Fix (proper): add a second small NFS share + mount on each nested host — more involved and rarely worth it for a lab.

`terraform destroy` aborts halfway

Storage policy "still associated with N entities"

Symptom:

```
Error: error while deleting policy with ID <uuid>: Profile <uuid> is
still associated with 6 entities.
```

…and then destroy stops, leaving haproxy/nfs/DVS/etc. intact.

Cause: the Supervisor disable destroy-provisioner didn't actually disable Supervisor (often because the `expect`-based SSH wcp-bounce silently failed). The 3 CP VMs are still around, still using the storage policy.

Fix: disable Supervisor manually, then re-run destroy:

```
SESSION=$(curl -sk -u $GOVC_USERNAME:$GOVC_PASSWORD -X POST https://$GOVC_URL/api/
        session | tr -d '"')
curl -sk -H "vmware-api-session-id: $SESSION" -X POST \
  https://$GOVC_URL/api/vcenter/namespace-management/clusters/<cluster-moref>?acti
        on=disable

# Wait until config_status reports GONE (~10-15 min):
while STATUS=$(curl -sk -H "vmware-api-session-id: $SESSION" \
  https://$GOVC_URL/api/vcenter/namespace-management/clusters/<cluster-moref> 2>/
        dev/null \
  | python3 -c "import
        json,sys;d=json.load(sys.stdin);print(d.get('config_status','GONE'))" 2>/
        dev/null); do
    echo "  $(date +%T) $STATUS"
    [ "$STATUS" = GONE ] && break
    sleep 30
done

terraform destroy -auto-approve
```

---

vCenter unreachable mid-apply

Symptom:

```
Error: error setting up new vSphere SOAP client: Post "https://vcenter…/sdk":
dial tcp <ip>:443: connect: operation timed out
```

Cause: transient network loss between your machine and vCenter.

Fix: re-run `terraform apply` / `destroy`. Terraform refreshes state and resumes. If it persists, check that `dig vcenter.skynetsystems.io` resolves correctly and that you can reach the vCenter UI.

Diagnostic command cheatsheet

```
# Source env
. /Users/ben/Repos/greylog/scripts/sv-env

# vCenter session token (for REST API one-shots)
SESSION=$(curl -sk -u "$GOVC_USERNAME:$GOVC_PASSWORD" -X POST https://$GOVC_URL/
        api/session | tr -d '"')
H="vmware-api-session-id: $SESSION"

# Supervisor cluster state
curl -sk -H "$H" https://$GOVC_URL/api/vcenter/namespace-management/clusters/<clus
        ter-moref> | python3 -m json.tool

# All VMs / port groups / datastores
govc find /Datacenter/vm -type m
govc find /Datacenter/network -type n
govc find /Datacenter/datastore

# Tags on a datastore
govc tags.attached.ls supervisor-storage

# HA advanced options on the supervisor cluster
govc cluster.info -dc=Datacenter Supervisor-Cluster

# Cloud-init status on a VM (after sshpass install)
sshpass -p '<pw>' ssh ubuntu@<vm-ip> 'cloud-init status; sudo tail -30 /var/log/
        cloud-init-output.log'
```